University of Pisa

Department of Computer Science

Fundamentals of Artificial Intelligence

BeerEX

the Beer EXpert system

A rule-based expert system that recommends a beer by taste and meal, reasoning under uncertainty with certainty factors and fuzzy logic

Donato Meoli

Technical report

Abstract

BeerEX (*the Beer EXpert system*) is a rule-based expert system, written in CLIPS, that recommends a beer style to drink according to the user's taste and meal. Through a guided dialogue it recognises the user scenario and preferences and, when relevant, the dish to be paired, and then ranks the most suitable beer styles. The system reasons under uncertainty through two interchangeable backends: a classic *certainty-factor* calculus and a *fuzzy-logic* engine whose membership functions are grounded in real style data (alcohol content, colour, bitterness) extracted from the CraftBeer.com 2017 Beer Styles Study Guide. The knowledge base further models, for every style, a full qualitative and serving profile and the authoritative food pairings published by the Brewers Association. The system can justify its behaviour —explaining *why* a question is asked and *how*, factor by factor, each recommendation was reached— and it is delivered both as a command-line program and as an asynchronous Telegram bot that keeps an isolated reasoning environment per user. This report covers the theoretical background, the analysis and conceptualization of the beer domain, the certainty-factor and fuzzy calculi, the modular CLIPS implementation, the validation against an authentic FuzzyCLIPS engine, and the usage and deployment of the system.

Contents

Chapter 1

Introduction

The aim of this work is to present a study concerning the development of an expert system, built in a rule-based programming environment, for the recommendation of a beer style according to the user's taste and to the meal to be paired.

Throughout the report we document the various design choices adopted during the definition phase, as well as the behaviour that the expert system exhibits during its execution.

Before discussing the project in detail, we provide a brief introduction to the features that characterise an expert system, to expert systems applied to the recommendation task, and to the handling of uncertain knowledge.

1.1 Expert Systems

One of the most actively developed areas in Artificial Intelligence is the study and design of expert systems, also referred to as *Expert Systems* or *Knowledge-Based Systems*. In the following sections we illustrate the features and the components that characterise an expert system.

1.1.1 Definition

The term *expert system* was introduced in 1977 by Feigenbaum, who also provided a precise definition:

> An expert system is a computer program that uses knowledge and reasoning techniques to solve problems that would normally require the help of a human expert. An expert system must be able to justify or explain why a particular solution was reached for a given problem.

An expert system must be capable of emulating the work of a human expert in a given domain. In particular, an expert system must be able to perform the same actions, provide the same judgements, and offer the same explanations as a human. This feature, however, should not obscure the ultimate goal of an expert system. Indeed, contrary to what one might think, its purpose is not to replace the expert, but rather to assist and help the user improve their performance.

A distinctive trait of this paradigm is the clear separation between *knowledge* —the facts and rules that describe the domain— and the *inference* mechanism that applies them. Conventional programming is designed for procedural data manipulation, whereas humans solve many problems through abstract, symbolic reasoning. Expert systems capture such reasoning by encoding it declaratively, so that the body of knowledge can be inspected, maintained and extended independently of the control flow.

The fundamental features of an expert system are the following:

- *Possession of specific knowledge*: it is important that an expert system possesses enough domain-specific knowledge. The quality of an expert system, in fact, does not depend only on the formalism or the inference technique adopted, but also on the knowledge that the program possesses;
- *Adherence of the domain to reality*: the domain in which an expert system operates must not be a simplified model of the real domain, but must represent it as it is, of course within certain limits, yet never resorting to excessively abstract models;
- *Reasoning capability*: an expert system must draw inferences and make decisions by exploiting the available knowledge base;
- *Capability of interacting with the external world*: an expert system must have a communication protocol with the external world. Such communication is necessary in order to retrieve the information required to carry on the reasoning and to provide explanations to the user;
- *Capability of explaining its own operation*: a good expert system must be able to make explicit and explain the decisions taken and the reasoning by which it reached a conclusion. This capability strengthens the degree of trust that the user places in the system;
- *Learning capability*: a feature that, while not common to all expert systems, is very interesting to consider. Just as a human expert learns new experiences and broadens their knowledge over time, an expert system should likewise be able to learn new knowledge.

A widely adopted way of representing declarative knowledge is the *production rule*, an `if-then` construct whose left-hand side states the conditions under which the right-hand side actions must be taken. Production systems are typically driven by *forward chaining*: the engine starts from the known facts and repeatedly fires the rules whose conditions are satisfied, deriving new facts until no further rule applies. CLIPS, the environment adopted in this work, implements this strategy efficiently by means of the *Rete* pattern-matching algorithm, which incrementally tracks which rules match the current contents of the working memory and thus avoids re-evaluating every rule on each cycle.

1.1.2 Components

Within an expert system it is possible to identify four fundamental components: the knowledge base, the inference engine, the user interface, and the working memory.

Knowledge Base

The knowledge base represents the set of information needed to address and solve a problem in a specific domain.

Such information is called *knowledge*, and it must be represented by means of a formalism that the machine can interpret (for example, Horn logic, description logic, modal logic, and so on).

Typically, the knowledge base is organised into two blocks:

- *Block of assertions or facts*: it represents the declarative knowledge related to a particular problem to be solved, that is, true facts introduced at the beginning of the consultation. In CLIPS the shape of a fact is declared through a `deftemplate`, while sets of initial facts are conveniently grouped within a `deffacts` construct;

- *Block of relations or rules*: it contains the relations among the elementary facts that are useful for building the body of knowledge. These are usually expressed in the form of production rules —the `defrule` construct in CLIPS— or according to other types of representation such as semantic networks, frames, scripts, and so on.

Inference Engine

The inference engine is a program able to deduce, starting from the knowledge base, the conclusions that constitute the solution of the problem in the domain in which the expert system operates.

The inference engine is able to identify the set of rules that are applicable at a given moment, to select the rule to apply among the candidate ones, and to execute the action indicated by the selected rule. In CLIPS the rules whose conditions are satisfied are placed on the *agenda*; when more than one rule is eligible, a *conflict resolution* strategy (based, for instance, on rule salience) decides the order in which they are fired.

User Interface

The user interface is responsible for facilitating the interaction between the user and the expert system. It can be built using user-friendly graphical interfaces or command-line interfaces. It is important that the interface makes the dialogue with the user as close as possible to a natural one.

Working Memory

The working memory is the component that allows the storage of newly observed evidence or of data useful for solving the problem during a specific working session. The content of this memory will therefore change throughout the entire execution session of the expert system, as facts are asserted and retracted in response to the user's answers and to the firing of rules.

1.1.3 Roles

In general, the roles that interact with an expert system are three:

- *Domain expert*: the actor who holds the knowledge and experience about a specific domain and is able to solve the problem at hand —in our case, the sommelier-like expertise that relates beer styles to tastes, occasions and food;
- *Knowledge engineer*: the actor who encodes the knowledge, provided by the domain expert, into a formalism that the expert system can understand;
- *User*: the individual who consults the expert system in order to benefit from it.

1.1.4 Building Environments

The construction of an expert system can be carried out using two different types of environment:

- *Skeleton environments*: these are environments derived from existing expert systems, from which the knowledge related to the application domain has been removed. Examples include Emycin, Kas, Expert, and so on;
- *Environment-type environments*: these are environments that provide a language for representing knowledge. Examples include Clips, Prolog, and so on.

Comparing the two types of environment, one can state that "Skeleton" systems are more immediate, but less flexible, since they are affected by the specialised architecture of the original product; "Environment"-type systems, on the other hand, have flexibility as their strength, while their weakness is the greater complexity.

BeerEX is built on CLIPS, a forward-chaining production-system tool that acts as a general-purpose shell for rule-based reasoning. Because knowledge is kept declarative, new styles, questions or food pairings can be added without touching the control flow, which is a decisive advantage for the maintainability of the system.

1.2 Expert Systems for Recommendation

A second field, beyond the classical diagnostic ones, in which the expert-system paradigm proves effective is that of *recommendation* and advisory tasks, and in particular within the food-and-beverage domain.

The craft-beer market is vast and fast-growing: in the United States alone there are thousands of breweries and well over a hundred recognised beer styles, with tens of thousands of brands. For the consumer this abundance translates into uncertainty: choosing a beer that suits one's taste, the season, or —above all— the food on the table requires expertise that most drinkers do not have. BeerEX addresses exactly this gap, acting as a pocket beer sommelier that, by asking a few simple questions, recommends in real time the most appropriate beer *style* for the situation, together with an explanation of the choice.

Such a task is a natural fit for an expert system. The domain is a bounded, symbolic one: the styles, their qualitative and serving profiles and the recommended food pairings can be enumerated and described declaratively. The knowledge involved is genuine expertise, comparable to that of a sommelier, which a non-expert user typically lacks. Finally, a recommendation is far more valuable when it is accompanied by an *explanation*: an expert system can make explicit *why* a question is asked and *how*, factor by factor, a particular style came to be suggested, thereby strengthening the user's trust in the advice. As with other advisory expert systems, the consultation allows a non-expert person to reach a decision that would otherwise lie beyond their level of knowledge and experience, while preserving and reusing the encoded expertise consistently across every consultation.

1.3 Handling Uncertain Knowledge

The cognitive mechanism of the human expert often relies on empirical rules acquired from their own experience and on data that are not completely certain.

An expert is able to reason even when the available information is uncertain or incomplete. This implies that conclusions obtained from knowledge held to be generally true, and by means of approximate reasoning, are not always absolutely correct.

This is precisely the case for a beer recommendation, which is rarely a black-or-white matter: tastes are graded, occasions overlap, and a given style may suit a dish more or less well rather than perfectly or not at all. The need therefore emerges for approaches that are able to replicate this approximate reasoning within the expert system.

There are several ways to represent and formalise uncertainty: they range from quantitative methods (certainty factors, Bayesian networks, belief functions, the fuzzy approach) to symbolic methods (non-monotonic logics). All these methods share the common feature of having to deal with the combination of evidence and the propagation of uncertainty from facts to conclusions.

BeerEX offers two interchangeable calculi for reasoning under uncertainty. The first is the classic *certainty-factor* model introduced by MYCIN, in which each piece of evidence and each conclusion carries a degree of belief, a *CF*, and the rules combine and propagate these factors.

The second is a *fuzzy-logic* engine whose membership functions are data-grounded, that is, derived from measurable beer parameters —alcohol content, colour and bitterness— so that linguistic judgements correspond to real style data. The user (or the deployer) can switch between the two backends, and the two can be compared on the same session. The details of both calculi are deferred to later chapters.

1.4 Scope of this report

The report follows the knowledge-engineering lifecycle: the analysis of the domain (Chapter 2), the identification of the task (Chapter 3), the conceptualization of the knowledge and of the two uncertainty models (Chapter 4), the CLIPS implementation and the Telegram delivery (Chapter 5), the ways to run the system (Chapter 6) and its validation against an authentic FuzzyCLIPS engine (Chapter 7).

Chapter 2

The beer domain

The following sections present the analysis of the domain over which BeerEX reasons, the dimensions along which beer is described, the taxonomy adopted to organize the styles, and the principles that govern the association between beer and food. A brief introduction to the notion of a beer *style* is followed by the distinction between styles and commercial brands; the measurable and perceptual parameters of a style are then defined and tabulated; the styles are classified into families; the three mechanisms of food pairing are described; and, finally, the documentary sources and the reproducible extraction pipeline that keep the knowledge base faithful to those sources are presented.

2.1 Beer styles versus brands

The domain modelled by BeerEX is that of beer *styles* rather than that of individual commercial *brands*. A style is an abstract, bounded category —"German-Style Pilsener", "Irish-Style Dry Stout", "Belgian-Style Tripel"— defined by a characteristic range of measurable parameters and a recognizable sensory profile, and codified by an authoritative body. A brand, by contrast, is a single product of a single brewery: an instance of a style, often itself drifting from batch to batch, and numbering in the tens of thousands worldwide. Choosing the style as the unit of the domain has three advantages. First, it is *finite and stable*: the catalogue of recognized styles changes slowly and is published, whereas the catalogue of brands is unbounded and volatile. Second, it is *described by data*: each style is associated with numeric ranges (alcohol, bitterness, colour, gravity, carbonation) and with a qualitative profile, so that the rules of the system have a firm, quantifiable substrate on which to operate. Third, it is the *level at which food pairing is meaningful*: the pairing guides associate dishes with styles, not with particular labels, because it is the style's flavour architecture, and not a brand's marketing, that determines whether a beer complements or clashes with a dish.

The original version of BeerEX encoded only a handful of symbolic attributes per style (alcohol, colour, flavour, fermentation, carbonation). The current knowledge base, defined in `clips/beer-styles.fct`, retains that symbolic backbone but enumerates **79** styles, each assigned to one of fifteen families and described by the symbolic slots `alcohol`, `color`, `flavor`, `fermentation`, `carbonation` and a reference `link` to the CraftBeer.com style page. This symbolic dataset is then enriched, style by style, by two further fact bases extracted directly from the source documents: the numeric ranges of `clips/beer-ranges.clp` (**73** of the 79 styles carry characteristic ranges; the remaining six are catch-all specialty or wild categories whose values span the entire scale and are therefore not informative) and the comprehensive qualitative, serving and pairing profiles of `clips/beer-data.clp` (**79** profiles and **73** food pairings). Throughout this report, "style" and "beer" are used interchangeably to denote one such entry.

2.2 Beer parameters

Each style is described along two complementary kinds of dimension: a set of *quantitative* parameters, which are continuous physical or chemical quantities measured during brewing or analysis, and a set of *qualitative* parameters, which are the perceptual descriptors recorded by tasters. To these is added the *serving* information —temperature and glassware— that completes the practical profile of a style.

2.2.1 Quantitative parameters

The quantitative parameters are the seven continuous quantities listed in Table 2.1. They are extracted as per-style closed intervals (a lower and an upper bound) from the source guide and stored in the `beer-range` deftemplate as the multislots `abv`, `ibu`, `srm`, `og`, `fg` and `co2`. The three of them that also serve as the system's principal linguistic axes —*colour* (SRM), *alcohol* (ABV) and *bitterness* (IBU)— are the ones along which the user expresses a preference and along which the fuzzy backend builds its membership functions.

- **Alcohol by volume (ABV)** is the fraction of the beverage's volume that is ethanol, the most familiar measure of strength.
- **International Bitterness Units (IBU)** quantify bitterness by the concentration of isomerized alpha-acids derived from the hops.
- **Standard Reference Method (SRM)** measures colour by light absorbance: low values denote pale, straw-coloured beers and high values deep brown to black ones.
- **Original gravity (OG)** is the density of the wort relative to water before fermentation; it reflects the amount of fermentable sugar available and hence the potential strength and body.
- **Final gravity (FG)** is the density after fermentation; the residual sugar it represents contributes to sweetness and body.
- **Apparent attenuation** is the proportion of the original extract consumed by the yeast, derived from OG and FG as $(\mathrm{OG} - \mathrm{FG})/(\mathrm{OG} - 1)$; it expresses how dry or how sweet the finished beer is.
- **Carbonation (CO_2 volumes)** is the amount of dissolved carbon dioxide, measured in volumes of gas per volume of beer; it governs the perceived effervescence.

Table 2.1 also reports, for each parameter, the overall extent actually spanned by the dataset, that is, the lowest lower bound and the highest upper bound observed across the styles that carry that parameter in `beer-ranges.clp`.

Table 2.1: The quantitative parameters of a beer style, with the overall range spanned by the dataset.

Parameter	What it measures	Unit	Dataset range
Alcohol by volume (ABV)	ethanol content, i.e. strength	% vol	2.8–14.2
Bitterness (IBU)	isomerized alpha-acid concentration from hops	IBU	3–100

continued on next page

Table 2.1 (continued)

Parameter	**What it measures**	**Unit**	**Dataset range**
Colour (SRM)	colour by light absorbance, pale to black	SRM	2–50
Original gravity (OG)	wort density before fermentation (fermentable sugar)	s.g.	1.028–1.140
Final gravity (FG)	density after fermentation (residual sugar)	s.g.	1.000–1.032
Apparent attenuation	fraction of extract fermented, derived from OG and FG	%	derived
Carbonation	dissolved CO_2, perceived effervescence	CO_2 vol.	1.0–4.5

2.2.2 Qualitative parameters

The qualitative parameters are the perceptual descriptors recorded, per style, in the `beer-profile` deftemplate of `clips/beer-data.clp`. Unlike the numeric ranges, they are free-text sensory characterizations taken verbatim from the source guide. The slots are the following.

- **Perceived alcohol** (`alcohol`): how evident the alcohol is on the palate, on a scale running through "Not Detectable", "Mild", "Noticeable", "Hot" and "Harsh" (e.g. "Not Detectable to Mild" for a Bohemian-Style Pilsener, "Noticeable to Hot" for an American Barley Wine).
- **Colour** (`colour`): the visual colour described in words (e.g. "Straw to Pale", "Copper to Reddish Brown", "Black"), complementing the numeric SRM range.
- **Clarity** (`clarity`): the transparency of the beer, from "Brilliant" and "Clear" through "Slight Haze" to "Hazy" and "Opaque".
- **Body** (`body`): the palate weight or mouthfeel, from "Drying" and "Soft" to "Moderate", "Mouth-Coating" and "Sticky".
- **Palate carbonation** (`palate-carbonation`): the perceived effervescence on the palate (e.g. "Low", "Medium", "Medium to High"), the sensory counterpart of the numeric CO_2 parameter.
- **Finish** (`finish`): the length and character of the aftertaste, from "Short" to "Long".
- **Hop aroma and flavour** (`hop`): the contribution of the hops to aroma, flavour and bitterness.
- **Malt aroma and flavour** (`malt`): the contribution of the malt, including the characteristic descriptors of the style (caramel, toast, biscuit, chocolate, coffee, and so on).

The yeast character, finally, is captured indirectly through the symbolic `fermentation` slot of `beer-styles.fct`, which records whether the style is *top*-fermented (ale), *bottom*-fermented (lager), spontaneously or mixed (*wild*), or a combination thereof. The profile also records the style's `country` of origin.

2.2.3 Serving

Two further slots of `beer-profile` describe how a style is meant to be served, and so complete its practical profile.

- **Temperature** (`temperature`): the recommended serving temperature, given as a Fahrenheit range. Lighter styles are served cooler —several pale and wheat styles at "40–45 °F"— and stronger, darker styles warmer (typically "50–55 °F"), so that their aromatics open up.
- **Glassware** (`glass`): the glass that best presents the style, drawn from a small vocabulary of vessels —among them the Tulip, the Nonic Pint, the Flute, the Snifter, the Goblet, the Vase and the Thistle— each suited to the style's strength, carbonation and aromatic intensity.

2.3 Beer style families

The 79 styles are organized into the **fifteen** families recorded in the `style` slot of `beer-styles.fct`. The classification follows the CraftBeer.com grouping, which combines the fundamental fermentation distinction (ale versus lager) with colour and strength: the families range from the pale, lightly fermented *Pilsener & Pale Lager* and *Pale Ale* groups, through the malt-forward *Brown Ale*, *Dark Lager*, *Bock*, *Porter* and *Stout* groups, to the high-strength *Strong Ale* and the aromatic *Belgian Style* and *Wild/Sour* groups, with the *India Pale Ale*, *Wheat Beer*, *Scottish-Style Ale* and *Hybrid Beer* families and a residual *Specialty Beer* family that collects flavoured, barrel-aged and otherwise non-canonical beers. Table 2.2 lists the fifteen families, their cardinality in the dataset, and a representative style for each, with that style's symbolic descriptors and the numeric ABV, IBU and SRM ranges actually recorded for it.

Table 2.2: The fifteen beer families, with a representative style and its recorded ABV (% vol), IBU and SRM ranges.

Family	**#**	**Representative style**	**Colour / flavour**	**ABV**	**IBU**	**SRM**
Pale Ale	5	American Pale Ale	amber, hoppy-bitter	4.4–5.4	30–50	6–14
India Pale Ale	3	American IPA	amber, hoppy-bitter	6.3–7.6	50–70	6–14
Pilsener & Pale Lager	5	German-Style Pilsener	pale, crisp-clean	4.6–5.3	25–40	3–4
Dark Lager	5	German-Style Dunkel	brown, dark-roasty	4.8–5.3	16–25	15–17
Brown Ale	3	American Brown Ale	brown, malty-sweet	4.2–6.3	25–45	15–26
Wheat Beer	6	German-Style Hefeweizen	pale, fruity-spicy	4.9–5.6	10–15	3–9
Belgian Style	7	Belgian-Style Tripel	pale/amber, fruity-spicy	7.1–10.1	20–45	4–9
Strong Ale	4	American Barley Wine	brown, hoppy-bitter	8.5–12.2	60–100	11–18
Bock	4	German-Style Doppelbock	brown/dark, malty-sweet	6.6–7.9	17–27	12–30
Porter	5	Robust Porter	dark, dark-roasty	5.1–6.6	25–40	30–50
Stout	5	Irish-Style Dry Stout	dark, dark-roasty	4.2–5.3	30–40	40–50

continued on next page

Table 2.2 (continued)

Family	#	Representative style	Colour / flavour	ABV	IBU	SRM
Scottish-Style Ale	2	Scotch Ale/Wee Heavy	brown/dark, malty-sweet	6.6–8.5	25–35	15–30
Hybrid Beer	6	California Common	amber/brown, hoppy-bitter	4.6–5.7	35–45	8–15
Wild/Sour	6	Contemporary Gose	pale, sour-tart-funky	4.4–5.4	10–15	3–3
Specialty Beer	13	American Black Ale	dark, hoppy-bitter	6.3–7.6	50–70	35–50

The table makes the structuring role of the three linguistic axes apparent. Colour (SRM) separates the pale families (Pilsener, Pale Ale, Wheat) from the dark ones (Porter, Stout) almost completely; alcohol (ABV) singles out the Strong Ale and Belgian families; and bitterness (IBU) distinguishes the hop-forward India Pale Ale and Strong Ale families from the malt-forward Bock, Dark Lager and Brown Ale families. These are precisely the gradients along which BeerEX's fuzzy membership functions are calibrated.

2.4 Food pairing

Beer-and-food pairing, as codified by the Brewers Association, is not arbitrary but rests on three complementary mechanisms, which the system encodes as the principles by which a dish is matched to a style.

- **Complement.** Shared or harmonious flavours reinforce one another, as when the roasted, coffee-like notes of a stout echo those of a chocolate dessert.
- **Contrast.** Opposing flavours balance one another, as when the sweetness of a malty beer offsets a savoury or spicy dish.
- **Cut.** The carbonation and bitterness of the beer cleanse the palate of fat and intensity, refreshing it between bites of a rich dish.

On the basis of these mechanisms, the source guides associate each style with the *cheese*, *entrée* and *dessert* that best match it. BeerEX records these associations in the `pairing` deftemplate of `clips/beer-data.clp`, with **73** of the styles carrying a triple of authoritative pairings. The system can therefore reason from a meal or dish back to a style: a request centred on a particular cheese, a particular entrée (roasted or grilled meats, seafood, sausages, spicy dishes) or a dessert is matched against these triples, and the matching styles are scored together with the measured attributes. Representative pairings drawn directly from the knowledge base are shown in Table 2.3.

Table 2.3: Representative food pairings, one per family, from `beer-data.clp`.

Style	Cheese	Entrée	Dessert
American Pale Ale	Mild or Medium Cheddar	Roasted or Grilled Meats	Apple Pie
American IPA	Blue Cheeses	Spicy Tuna Roll	Persimmon Rice Pudding

continued on next page

Table 2.3 (continued)

Style	Cheese	Entrée	Dessert
German-Style Pilsener	White Cheddar	Salads	Shortbread Cookies
German-Style Dunkel	Washed-Rind Munster	Sausages	Candied Ginger Beer Cake
American Brown Ale	Aged Gouda	Vegetables	Pear Fritters
German-Style Hefeweizen	Chèvre	Seafood	Key Lime Pie
Belgian-Style Tripel	Triple Creme	Roasted Turkey	Caramelized Banana Creme Brulee
American Barley Wine	Strong Blue Cheeses	Beef Cheek	Rich Desserts
German-Style Doppelbock	Strong Cheeses	Ham	German Chocolate Cake
Robust Porter	Gruyère	Roasted or Grilled Meats	Chocolate Peanut Butter Cookies
Irish-Style Dry Stout	Irish Cheddar	Ham	Chocolate Desserts
Scotch Ale/Wee Heavy	Pungent Cheeses	Game	Creamy Desserts with Fruit
California Common	Feta	Pork Loin	Bread Pudding
Contemporary Gose	Queso Fresco	Watermelon Salad	Greek Yogurt Lemon Mousse
American Black Ale	Blue Cheeses and Aged Gouda	Grits	Chocolate Truffles

The pairings illustrate the three mechanisms at work: the roasted Irish-Style Dry Stout matched to chocolate desserts *complements* like with like; the malty German-Style Doppelbock matched to ham *contrasts* sweet against savoury; and the highly carbonated, bitter American IPA matched to a spicy tuna roll *cuts* through heat and fat. The same data also support meal- and dish-oriented reasoning of the kind a user typically begins from —for instance pizza, or a cheese course— since dishes such as roasted or grilled meats, seafood, sausages and the various cheeses recur across families and can be used as entry points into the style space.

2.5 Sources and extraction

The knowledge base is built entirely from authoritative material published by CraftBeer.com and the Brewers Association, so that every fact in the system is traceable to a published source.

- the *2017 Beer Styles Study Guide*, which provides, per style, the numeric ranges (ABV, IBU, SRM, OG, FG, CO_2 volumes) and the full qualitative and serving profile;
- the *Beer & Food Course* and the accompanying food-pairing chart, which provide the three pairing mechanisms (complement, contrast, cut) and the authoritative cheese / entrée / dessert associations.

Rather than transcribing this material by hand, BeerEX mines it with two reproducible scripts, so that the knowledge base stays faithful to —and is regenerable from— the documents.

- `scripts/extract_ranges.py` parses the Study Guide PDF (via `pdftotext`), locates each style page, reads off the six numeric ranges, and matches each guide entry against the 79 styles of `beer-styles.fct` by fuzzy string matching. It deliberately discards catch-all spans —ranges too wide to be characteristic, as listed by the specialty and wild

categories— so that only informative ranges enter the engine. It writes the auditable table `data/beer-style-ranges.csv` and the CLIPS facts `clips/beer-ranges.clp`.

- `scripts/extract_kb.py` parses both the Study Guide PDF, for the eleven qualitative, serving and origin fields, and the pairings spreadsheet, for the cheese / entrée / dessert triples, again aligning every record to the 79 styles. It writes the comprehensive table `data/beer-styles.csv` and the CLIPS facts `clips/beer-data.clp` (the `beer-profile` and `pairing` deftemplates and their deffacts).

Because the source documents are kept outside version control, the generated CSV tables and `.clp` fact bases serve as the auditable, committed record of the extraction; re-running the scripts against the same documents reproduces them exactly. This pipeline is what allows BeerEX to replace the hand-assigned weights of its first version with a data-grounded knowledge base, and it is on this foundation that the certainty-factor rules and the fuzzy backend described in the following chapters are built.

Chapter 3

Task identification

In this chapter we describe the phases that led to the acquisition of the domain knowledge, the identification of the requirements to be implemented, and the definition of the reasoning strategy.

The task of BeerEX is, given a user, to recommend the beer style(s) that best suit that user in the current situation. The recommendation is grounded on three layers of information, acquired through a guided dialogue: the *scenario* of the user, their personal *preferences*, and the *meal* they are about to enjoy. From the answers, the expert rules infer a desired beer profile, and a selection stage ranks the candidate styles by how well they match it, reasoning under uncertainty through one of two interchangeable calculi.

3.1 Knowledge Acquisition

The acquisition of the knowledge of the application domain was carried out by drawing on two complementary kinds of source:

- *Authoritative guides*: the beer-style and food-pairing material published by CraftBeer.com on behalf of the Brewers Association [6, 7], namely the *Beer Styles Study Guide* (the descriptive and numeric characterisation of each style: colour, alcohol, bitterness, fermentation, flavour) and the food-pairing tables (the recommended style for each food).
- *Domain heuristics*: the rules of thumb that relate the user's scenario—age, drinking habit, smoking habit, season—to the strength and flavour profile of an enjoyable beer, distilled into the *why* rationales attached to the corresponding questions.

Unlike the descriptive guides, which characterise the styles, the recommendation task requires eliciting the *user's* situation. The system therefore acquires the knowledge needed to advise the user through a dialogue organised in three layers:

1. the **user scenario**—age, whether a regular beer drinker, whether smoking, and the season—factors that shift, weakly, the expected strength and flavour of an enjoyable beer;
2. the **personal preferences**—preferred colour, alcohol level, carbonation and flavour;
3. the **meal**, when the user is about to eat—main component, type and cooking method, refined through increasingly specific questions down to a single dish.

Each answer asserts a fact in working memory, from which the expert rules infer *evidence* for the desired beer profile. The questions handled by the system, grouped by layer, together with the attribute each elicits and the set of admissible answers, are classified in Table 3.1.

Table 3.1: Classification of the question groups handled by the expert system, with the attribute each elicits and its admissible answers.

Layer / group	**Attribute elicited**	**Admissible answers**
Scenario (asked in random order)		
Age	`which-age`	18–23, 24–29, ≥30.
Regular drinker	`regular-beer-drinker`	yes, no.
Smoking habit	`smoker`	yes, no.
Season	`which-season`	autumn, spring, summer, winter.
Preferences (asked in random order)		
Colour	`preferred-color`	pale, amber, brown, dark, don't mind.
Alcohol	`preferred-alcohol`	low, mild, high, very high, don't mind.
Carbonation	`preferred-carbonation`	low, medium, high, don't mind.
Flavour	`preferred-flavor`	clean, sweet, bitter, roasty, fruity, spicy, sour, don't mind.
Meal (depth-first chain)		
Main meal	`main-meal`	pizza, entrée, cheese, dessert, other.
Pizza topping	`pizza-topping`	classic, meat, vegetables, cheese, other.
Spicy meat	`meat-topping-is-spicy`	yes, no.
Roasted vegetables	`vegetables-topping-are-roasted`	yes, no.
Entrée component	`which-entrée`	grain, legumes, fish, meat, vegetables, fats, other.
Fish type	`which-fish`	shellfish, other.
Mild shellfish	`shellfish-is-mild`	yes, no.
Shellfish type	`which-shellfish`	mussels, oysters, other.
Meat type	`which-meat`	rich, poultry, game, other.
Meat cooking method	`meat-cooking-method`	braised, grilled, roasted, other.
Rich meat	`which-rich`	beef, lamb, pork, other.
Beef type	`which-beef`	bresaola, other.
Pork cut	`which-pork`	loin, prosciutto, speck, mortadella, sausage, other.
Sausage type	`which-sausage`	capocollo, soppressata, salame piccante, other.
Poultry type	`which-poultry`	chicken, other.
Game type	`which-game`	wild, birds.
Vegetables type	`which-vegetables`	root, other.
Vegetables cooking method	`vegetables-cooking-method`	grilled, other.
Other vegetables	`which-other-vegetables`	mushrooms, other.
Fats type	`which-fats`	vegetable, other.
Cheese style	`which-cheese-style`	fresh, semi-soft, firm/hard, blue, natural rind, washed rind, don't know.

continued on next page

Layer / group	Attribute elicited	Admissible answers
Fresh cheese	`which-fresh-cheese`	Mascarpone, Chèvre, Feta, Cream Cheese, other.
Semi-soft cheese	`which-semi-soft-cheese`	Mozzarella, Colby, Havarti, Monterey Jack, other.
Firm/hard cheese	`which-firm/hard-cheese`	Gouda, Cheddar, Swiss, Parmesan, other.
Gouda type	`which-type-of-Gouda`	aged, smoked, other.
Cheddar colour	`which-color-of-Cheddar`	white, other.
Cheddar seasoning	`which-Cheddar-seasoning`	mild, medium, aged, other.
Swiss type	`which-type-of-Swiss`	Emmental, Gruyère, other.
Blue cheese	`which-blue-cheese`	Stilton, other.
Natural-rind cheese	`which-natural-rind-cheese`	Brie, Camembert, Triple Crème, Mimolette, Stilton, other.
Washed-rind cheese	`which-washed-rind-cheese`	Taleggio, other.
Dessert type	`which-dessert`	creamy, fruit, chocolate, other.
Fruity creamy dessert	`creamy-dessert-with-fruit`	yes, no.
Chocolate type	`which-chocolate`	white, milk, semisweet, bittersweet, unsweetened/bitter, don't know.

3.2 Requirements

In the following sections we provide a brief description of the requirements that the system must satisfy and of the features to be implemented.

Programming Language

The expert system is implemented in CLIPS (the *C Language Integrated Production System*), a forward-chaining production-system tool well suited to rule-based expert systems.

By using CLIPS as the programming language, the knowledge about the identified objects and relations can be expressed declaratively, in the form of facts and production rules, while the questions to be posed to the user—needed to derive further information useful to the inference process—are themselves encoded as rules that assert the dialogue state. The inference engine matches the rules against working memory by means of the *Rete* algorithm, which makes the pattern matching efficient even as the number of facts and rules grows.

We rely on a stock, modern CLIPS (the 6.4.x line), without forking the engine, so that the knowledge base remains portable. For the deployment as a Telegram bot, the engine is driven from Python through `clipspy`, the official CLIPS bindings, which embed the very same engine used by the command-line interpreter, so that the behaviour of the bot and of the CLI coincide.

Two behavioural features required by the system deserve a dedicated discussion: the handling of uncertainty and an explanation module. They are discussed below.

Handling of Uncertainty

The first behavioural feature concerns the handling of uncertainty.

A categorical kind of reasoning, in which every element can only be either true or false, turns out to be too restrictive for the task at hand. The suitability of a beer is a matter of degree: a given scenario, preference or dish does not single out one style, but rather makes a whole profile—a colour, an alcohol level, a flavour—*more or less* desirable. The evidence

supporting the desired profile is therefore inherently graded, and the system must be able to accumulate it and to weigh it against the measured characteristics of each candidate style.

From these premises, the need emerges to move away from the categorical nature of assertions and to attach a degree of confidence to every piece of evidence and to every recommendation derived from it.

BeerEX provides two interchangeable calculi for reasoning with this graded evidence, selectable through a single backend flag. The first is the *certainty-factor* (CF) calculus of MYCIN, in which each piece of evidence carries a certainty factor in $[-1, 1]$ and contributions to the same conclusion are merged with the MYCIN combination function. The CF weights, however, are assigned by the expert and the source guides do *not* quantify them; this intrinsic arbitrariness is precisely what motivates the second calculus, a *fuzzy-logic* backend whose membership functions are derived from data—by crossing the symbolic labels of the dataset with the numeric ranges of the guide—so that the matching of the candidate styles against the desired profile rests on objective, data-grounded quantities rather than on hand-assigned weights.

The two backends are described in depth in the chapter devoted to knowledge conceptualisation; here it suffices to note that both consume the same evidence and differ only in how they combine it and match it to the styles.

Explanation Module

The second behavioural feature concerns the implementation of an explanation module.

In an expert system, the explanation capability is of fundamental importance. Users tend not to trust an artificial system slavishly, nor to accept blindly a recommendation produced in an uncontrollable way. The capability to justify both the questions and the result increases the trust the user places in the advice of the system and enables them to discover a possible flaw in the reasoning.

In keeping with this requirement, the expert system provides explanations on two distinct occasions. During the dialogue, at every question the user may ask for *help*—an explanation of the question, sometimes with illustrative images—or for *why* the question is being asked, so as to suit different levels of expertise. At the end of the consultation, the system returns a twofold explanation of the result: a narrative that recounts how the user scenario, the preferences and the meal shaped the desired profile, and—in fuzzy mode—a per-beer rationale that lists the factors that matched each recommendation and how strongly.

3.3 Reasoning Strategy

As far as the type of inference is concerned, the choice fell on forward chaining, the inference natively provided by CLIPS: starting from the known facts contained in working memory, the production rules are applied forward in order to deduce new facts, until no further rule is applicable.

The dialogue drives the inference in three layers. The *scenario* and *preference* questions are independent of one another and are therefore asked in random order, by setting the conflict-resolution strategy to `random` while their rules are eligible to fire. The *meal* questions, on the contrary, form a depth-first chain: each answer enables the next, more specific question, narrowing the dialogue from the main course down to a single dish—so the strategy is switched to `depth` once the meal phase begins.

Each answer asserts a fact in working memory through the `relation-asserted` of the question. The knowledge rules, which sit at a medium-low salience, then fire on these facts and assert the *evidence* for the desired profile, in the form `(attribute (name best-`*X*`) (value` *v*`) (certainty` *w*`))`, for the colour, alcohol, carbonation, flavour and fermentation axes, together

with direct suggestions of a specific style or beer name and the narrative fragments of the explanation.

Because the dialogue allows the user to go *back* and revise a previous answer, the working memory must be kept consistent with the answers actually given. When the user goes back, the dialogue state machine retracts the fact asserted by the revised question and refreshes the affected rules, so that everything inferred from the superseded answer is removed and re-derived from the new one; in random mode the question rules are refreshed so they may be asked again, and if the consultation had already reached its final state the selection rules are refreshed as well.

Finally, once all the evidence has been gathered, a selection stage at low salience ranks the styles. Contributions to the same `best-*` value are merged with the certainty-factor combination function; each candidate style is then scored against the desired profile—by accumulating the certainties of the matching axes in CF mode, or by a data-grounded weighted-mean satisfaction in fuzzy mode—and the most appropriate styles are returned, each with its confidence and a serving hint, followed by the explanation.

The reasoning strategy introduced here is taken up again and documented in depth in the chapters devoted to the description of the inference engine.

Chapter 4

Knowledge conceptualization

In this chapter we present an analysis of the process that led to expressing the domain knowledge in logical form. In particular, we discuss the objects that were identified, together with the relations and properties of those objects, and we give a conceptual overview of the two complementary models through which the system reasons under uncertainty. The detailed calculus that underpins the two models—the certainty-factor combination, the trapezoidal membership functions, the centroid defuzzification and the weighted-mean scoring—is deferred to the Implementation chapter, which owns the equations and their derivations; here we remain at the conceptual level.

The conceptualization is organised around the following steps, which structure the reasoning from the elementary dialogue with the user up to the final recommendation:

(I) *Domain objects* —the beer styles and the named beers the system can recommend, each carrying the symbolic descriptors (colour, alcohol, flavour, fermentation, carbonation) over which the qualitative reasoning is performed, encoded by the `beer` template;

(II) *Data-grounded descriptions* —the authoritative per-style sensory profiles, numeric ranges and food pairings extracted from the CraftBeer 2017 Beer Styles Study Guide, encoded by the `beer-profile`, `beer-range` and `pairing` templates;

(III) *Attributes and evidence* —the graded pieces of evidence about the user's ideal profile, accumulated on the colour, alcohol, flavour, fermentation and carbonation *axes*, encoded by the `attribute` template in the canonical *best-** form;

(IV) *The dialogue / evidence model* —the conversational machinery that elicits the user's desires through scenario, preference and meal questions, tracks the conversation and overlays localised labels, encoded by the `UI-state`, `state-list`, `translation` and `meal-keyword` templates;

(V) *The two uncertainty models* —the MYCIN certainty-factor model and the data-grounded fuzzy-logic model, the latter introducing linguistic variables, desires and direct suggestions, encoded by the `ling-var`, `ling-term`, `desire`, `desire-cat`, `suggest-name` and `suggest-style` templates;

(VI) *The output* —the ranked beer recommendations and their data-grounded rationale, encoded by the `attribute` *beer* facts and the `beer-why` template.

4.1 Objects

The conceptualization phase brought several objects to light. In Table 4.1 we report them as the *deftemplates* through which they are represented in working memory, describing what

each represents and listing its real slots together with their type, so as not to weigh down the discussion with the enumeration of every individual fact.

Table 4.1: Objects identified during the conceptualization phase, with their real slots.

Object	**Description**	**Attributes (slots / type)**
`beer`	A named beer belonging to one of fifteen recognised styles, described by the symbolic descriptors over which the qualitative reasoning matches.	`style` (STRING, allowed-strings); `name` (STRING); `alcohol`, `color`, `flavor`, `fermentation`, `carbonation` (multislot SYMBOL, each with allowed-symbols); `link` (STRING).
`beer-profile`	The authoritative per-style sensory profile extracted from the CraftBeer 2017 guide: descriptors, palate, hop/malt character and serving advice.	`name` (STRING); `alcohol`, `colour`, `clarity`, `body`, `palate-carbonation`, `finish`, `hop`, `malt`, `glass`, `temperature`, `country` (STRING).
`beer-range`	The per-style numeric ranges extracted from the same guide; these data ground the fuzzy membership functions on the measurable axes.	`name` (STRING); `abv`, `ibu`, `srm`, `og`, `fg`, `co2` (multislot).
`pairing`	The authoritative food pairing for a style (cheese, entrée and dessert), matched against the foods named by the user.	`style` (STRING); `cheese`, `entree`, `dessert` (STRING).
`attribute`	A graded piece of evidence about the desired profile, in the canonical *best-** form (e.g. `best-color`, `best-alcohol`, `best-flavor`, `best-style`, `best-name`), and the final recommendation facts (`name beer`).	`name`; `value`; `certainty` (FLOAT, range $[-1, 1]$, default 1.0).
`ling-var`	A linguistic variable: one measurable axis together with the bounds of its universe of discourse.	`axis`; `from`; `to`.
`ling-term`	A linguistic term of an axis, represented as a trapezoidal membership function $a \leq b \leq c \leq d$ (shoulders when $a{=}b$ or $c{=}d$), with breakpoints derived from the data distributions.	`axis`; `term`; `a`, `b`, `c`, `d`.
`desire`	A weighted preference for a linguistic term on a continuous axis, derived from the *best-** evidence in fuzzy mode.	`axis`; `term`; `weight` (default 1.0).

continued on next page

Object	Description	Attributes (slots / type)
`desire-cat`	A weighted preference for a value on a categorical (symbolic) slot of a beer, such as `flavor` or `carbonation`.	`slot`; `value`; `weight` (default 1.0).
`suggest-name`	A direct, weighted suggestion of a specific beer name (e.g. produced by a meal pairing).	`name`; `weight` (default 1.0).
`suggest-style`	A direct, weighted suggestion of a specific beer style.	`style`; `weight` (default 1.0).
`UI-state`	A single turn of the dialogue: the message shown to the user, its help/why text, the admissible answers and the recorded response, together with the relation it asserts and its position in the conversation.	`id` (gensym default); `display`; `help`; `why`; `relation-asserted` (default `none`); `valid-answers` (multislot); `response` (default `none`); `state` (default `middle`); `translated` (default `FALSE`).
`state-list`	The bookkeeping of the conversation: the current turn and the sequence of turns visited, enabling navigation and clean-up.	`current`; `sequence` (multislot).
`translation`	A gettext-style localisation overlay: the English string is the source key, the Italian string is the optional overlay (with English fallback).	`en` (STRING); `it` (STRING).
`meal-keyword`	A food the user mentioned during the dialogue, matched against the authoritative `pairing` facts.	`text`.
`beer-why`	The per-beer rationale produced by the fuzzy backend (which factors matched and how strongly), used to build a data-grounded explanation.	`name`; `text`.

4.2 Relations and Properties

Table 4.2 reports the relations among the objects and the properties that link them. The first group describes how the data-grounded descriptions qualify a beer or style; the second how evidence is accumulated and turned into desires; the third how the dialogue is modelled and localised.

Table 4.2: Relations and properties of the objects.

Relation	**Description**
A `beer` *belongs to* a `style`	Each beer carries a `style` drawn from the fifteen recognised styles and is described by its symbolic *axes* (colour, alcohol, flavour, fermentation, carbonation), over which both backends match candidates.
A `beer-profile` *describes* a beer	Joined by `name`, the profile supplies the authoritative sensory description and the serving hints (glass, temperature) appended to a recommendation.
A `beer-range` *measures* a beer	Joined by `name`, the numeric ranges (ABV, IBU, SRM, ...) provide the crisp value—the midpoint of the range—fed to the fuzzy membership functions on the colour, alcohol and bitterness axes.
A `pairing` *recommends* a style	Joined by `style`, the authoritative cheese/entrée/dessert pairing is matched against the `meal-keyword` facts to grant a pairing bonus.
An `attribute` *carries best-* evidence with a certainty*	Each *best-** attribute records a desired value on one axis (or a direct style/name suggestion) together with its certainty in $[-1, 1]$; rules that assert the same value on the same axis are merged into a single accumulated certainty.
A `ling-var` *spans* an axis; a `ling-term` *partitions* it	Each measurable axis (colour ← SRM, alcohol ← ABV%, bitterness ← IBU) has a universe of discourse and is covered by overlapping trapezoidal terms whose breakpoints are derived from the data distributions.
A `desire` / `desire-cat` *weights* a preference	In fuzzy mode the *best-** evidence is translated into desires: a `desire` weights a linguistic term on a continuous axis, while a `desire-cat` weights a value on a categorical slot; both carry a weight in $[-1, 1]$.
A `suggest-name` / `suggest-style` *points to* a candidate	A direct, weighted suggestion of a specific beer name or style, typically produced by a meal pairing, contributing to that candidate's score.
A `beer-why` *explains* a recommendation	Joined by `name`, it records which desires the beer satisfied and how strongly, so that the final explanation is data-grounded.

continued on next page

Relation	Description
A `UI-state` *asserts* a relation and *advances* the dialogue	Each turn shows a message, offers `valid-answers`, records the `response` and asserts the corresponding fact; its `state` (initial / middle / final) marks the phase of the conversation.
A `state-list` *orders* the conversation	It keeps the `current` turn and the `sequence` of visited turns, enabling navigation through the dialogue and final clean-up.
A `translation` *overlays* a label	Keyed on the English source string, it provides the localised (Italian) text, falling back to English when no overlay exists.
A `meal-keyword` *matches* a pairing	Each food named by the user is compared against the cheese/entrée/dessert text of the `pairing` facts to reward styles whose authoritative pairing matches the user's meal.

4.3 The two reasoning models (overview)

Both backends—selected by the `?*backend*` global (`cf` or `fuzzy`)—consume the same *best-** evidence accumulated on the colour, alcohol, flavour, fermentation and carbonation axes during the dialogue. They differ only in how that evidence is combined and matched against the candidate beers. We give here a conceptual overview; the formal calculus is detailed in the Implementation chapter.

4.3.1 Certainty factors

The first model is the classical MYCIN scheme. Every piece of evidence is a graded judgement carrying a certainty factor in $[-1, 1]$, ranging from full disbelief through ignorance to full belief. When several rules independently assert the same value on the same axis, their certainties are merged by the MYCIN combination function, which accumulates concordant evidence towards the extremes while letting discordant evidence cancel. A beer is then scored by accumulating the certainties of the axes whose symbolic value it matches, and the highest-scoring candidates are returned. The model is transparent and faithful to expert practice, but the weights are assigned by the expert and are not quantified by the source guides; this intrinsic arbitrariness, and the coarse ties it produces, is precisely what motivates the fuzzy alternative.

4.3.2 Fuzzy logic

The second model replaces the hand-assigned certainties on the *measurable* axes with objective, data-grounded memberships. The measurable axes are modelled as *linguistic variables*—colour over SRM, alcohol over ABV% and a new bitterness axis over IBU—each covered by overlapping linguistic terms encoded as trapezoidal membership functions. Crucially, the breakpoints are *derived from data*: crossing the symbolic labels of the dataset with the numeric ranges of the guide yields the empirical distribution of each term (validated against the p25 / median / p75 quantiles), from which the trapezoids are built. The breakpoints currently in use are summarised in Table 5.2 as a forward reference.

Carbonation, by contrast, stays symbolic: the guide's CO_2 volumes do not separate *medium* from *high*, so a data-driven membership function would not be justified. In fuzzy mode the *best-**

Table 4.3: Data-grounded trapezoidal breakpoints of the linguistic terms (forward reference; derivation in the Implementation chapter).

axis	source	terms (trapezoid breakpoints a, b, c, d)			
colour	SRM	pale (0,0,6,9)	amber (6,9,11,14)	brown (11,14,21,27)	dark (21,27,50,50)
alcohol	ABV%	n.d. (2,2,4.5,5.2)	mild (4.5,5,6,6.6)	noticeable (6,6.6,8,8.6)	harsh (8,8.6,15,15)
bitterness	IBU	low (0,0,20,25)	medium (20,25,30,35)	high (30,35,70,70)	

evidence is translated into *desires*—weighted preferences on the linguistic axes— categorical flavour and carbonation desires, and direct style/name suggestions arising from the meal. Each beer is then ranked by a normalised weighted-mean satisfaction score: the membership of the beer's measured value in each desired term, weighted by the desire and normalised by the total weight, with a beer missing data on a desired axis penalised rather than exempted. A further term boosts styles whose authoritative CraftBeer pairing matches a food named by the user. Here the gain is real: the memberships are objective, so the fuzzy ranking grades the candidates finely where the certainty-factor model can only produce coarse ties. Membership evaluation, the fuzzy set operations and the centroid defuzzification are provided by a reusable, agnostic pure-CLIPS fuzzy library; the corresponding equations and the equivalence with the original FuzzyCLIPS primitives are given in the Implementation chapter.

Chapter 5

Implementation

This chapter describes how BeerEX is realised: the tools adopted, the modular architecture of the system, the representation of the knowledge base, the mechanisms of the inference engine —knowledge acquisition, the management of uncertainty under the two backends, and the explanation of the reasoning— and the bilingual user interface shared by the command-line program and the Telegram bot. The formal calculus introduced conceptually in the previous chapter —the MYCIN certainty-factor combination, the trapezoidal membership functions, the centroid defuzzification and the weighted-mean scoring— is owned here, together with its grounding in the source files.

5.1 Tools Used

BeerEX is implemented in *CLIPS*, the C Language Integrated Production System (NASA/Johnson Space Center), a mature and free forward-chaining production-rule language. CLIPS was chosen because the recommendation knowledge is naturally expressed as *rules* that fire on asserted facts, and because its `deftemplate`/`deffacts` machinery provides a clean working-memory model for the data-grounded knowledge base. The features that CLIPS does not provide natively —reasoning under uncertainty and fuzzy inference— are implemented explicitly on top of the language, as described below.

The system is delivered in two forms, sharing the same `.clp` sources:

- a **command-line program** (`BeerEX.clp`), run by a CLIPS 6.4.2 interpreter built locally from source by `scripts/build-clips.sh` (which fetches and compiles the official CLIPS sources into `~/.local/bin`). The same interpreter runs the `.clp`-based tests;
- an asynchronous **Telegram bot** (`BeerEX.bot.py`), built on *python-telegram-bot* v21, that reaches the engine through *clipspy*. clipspy bundles its own CLIPS 6.4.x, so the bot needs no separately built interpreter.

The fuzzy mathematics is supplied by a reusable, domain-agnostic pure-CLIPS library (`dmeoli/FuzzyCLIPS`), vendored as a *git submodule* under `third_party/`; this keeps BeerEX on stock, modern CLIPS while still offering genuine fuzzy reasoning (Section 5.4). The project ships a `Dockerfile` and a `docker-compose` file, and a CI workflow that lints, runs the test suite and publishes the image to the GitHub Container Registry (GHCR).

5.2 System Architecture

The system is organised by *responsibility* rather than as a single monolithic file. This separation mirrors the conceptual architecture of an expert system (knowledge base, inference engine, user interface) and keeps each concern independently readable and testable. Table 5.1 lists the

modules; `BeerEX.clp` (CLI) and the `CORE_FILES` list of `BeerEX.bot.py` (bot) collect them and fix the load order.

Table 5.1: Modular organisation of BeerEX (file → responsibility).

Module	**Responsibility**
`BeerEX.clp`	CLI bootstrap: load order, the `ask-question` / `next-UI-state` REPL driver, `(reset)` / `(run)`.
`beerex.clp`	Core: templates (`UI-state`, `attribute`, `beer`, ...), globals (`?*backend*`, `?*lang*`, salience), `combine-CFs`, the *CF* selection rule and the print/clean-up rule, `get-explanation`, the `T` localisation function.
`dialog.clp`	Dialogue state machine: `ask-question` halt, `handle-next`/`handle-prev` navigation and retraction, `translate-ui`.
`beer-questions.clp`	The questions posed to the user (scenario, preference and meal questions).
`beer-knowledge.clp`	Expert rules that turn answers into *best-** evidence with certainties.
`beer-styles.fct`	Symbolic style/beer dataset (the `beer` facts).
`beer-ranges.clp`	Per-style numeric ranges (ABV, IBU, SRM, ...) —the data behind the fuzzy memberships.
`beer-data.clp`	Full per-style sensory profile and the food pairings.
`beer-fuzzy.clp`	Fuzzy backend: linguistic variables/terms, membership, scoring/ranking, the per-beer rationale and the *CF*→fuzzy bridge.
`beerex-fuzzy.clp`	Wires the fuzzy backend into the selection stage (`generate-beers-fuzzy`).
`labels-it.clp`	Optional Italian overlay (gettext-style `translation` facts).
`third_party/FuzzyCLIPS/fuzzy.clp`	Reusable pure-CLIPS fuzzy library (submodule): membership constructors, set operators, defuzzifiers, Mamdani.
`BeerEX.bot.py`	Telegram bot: one isolated `clips.Environment` per chat, dialogue driver, inline keyboards, `/cf` `/fuzzy` `/en` `/it`.

A single global, `?*backend*` (`cf` | `fuzzy`), selects the reasoning strategy at the low-salience selection stage, leaving the dialogue and the knowledge rules untouched; both backends emit the same `(attribute (name beer) ...)` result facts, so printing and explanation are shared.

5.3 Knowledge Base

The knowledge base is represented through CLIPS `deftemplate`s and `deffacts`, split across the data files of Table 5.1. Three families of facts coexist:

(I) **Domain facts** —the `beer` facts in `beer-styles.fct` carry the symbolic descriptors over which the qualitative reasoning matches: `style`, `name` and the multislot axes `alcohol`, `color`, `flavor`, `fermentation`, `carbonation` (each constrained by `allowed-symbols`);

(II) **Data-grounded descriptions** —the `beer-range` facts (`beer-ranges.clp`: ABV, IBU, SRM, OG, FG, CO_2 ranges) and the `beer-profile` and `pairing` facts (`beer-data.clp`);

(III) **Evidence** —the `attribute` facts asserted during the dialogue, in the canonical *best-** form (`best-color`, `best-alcohol`, `best-flavor`, `best-style`, `best-name`, ...), each with a certainty in $[-1, 1]$.

The questions (`beer-questions.clp`) and the knowledge rules (`beer-knowledge.clp`) are organised in parallel: each question asserts a `UI-state` whose `relation-asserted` names the relation it elicits (e.g. `which-season`), and the corresponding knowledge rule fires on that relation to assert the *best-** evidence.

The data-grounded facts are not written by hand but produced by two reproducible Python scripts that mine the CraftBeer 2017 sources, so the knowledge base can be regenerated from the documents and audited:

- `scripts/extract_ranges.py` parses the numeric style ranges from the *Beer Styles Study Guide* PDF and maps them onto the named beers, emitting both an auditable `data/beer-style-ranges.csv` and the engine fact file `clips/beer-ranges.clp`;
- `scripts/extract_kb.py` parses the qualitative descriptors, palate, hop/malt and serving profile, and the cheese/entrée/dessert food pairings (from the spreadsheet), emitting `data/beer-styles.csv` and `clips/beer-data.clp`.

5.4 Inference Engine

The inference engine derives the recommendation from the facts stated by the user. It comprises four concerns: the acquisition of evidence, the management of uncertainty under the *CF* backend, its fuzzy alternative, and the explanation of the reasoning.

Knowledge Acquisition. Evidence is gathered through a uniform pipeline: question → fact → rule → *best-** evidence. A question rule asserts a `UI-state` that records the relation it elicits; when the user answers, the dialogue machinery (Section 5.5) asserts the relation as a plain fact (e.g. `(which-season winter)`); a knowledge rule then matches that fact and asserts the graded *best-** attributes. For instance the winter-scenario rule contributes several weighted preferences across the axes:

```
(defrule determine-best-beer-attributes-if-which-season-is-winter
   (declare (salience ?*medium-low-priority*))
   (which-season winter)
   =>
   (assert (attribute (name best-color) (value dark) (certainty .05)))
   (assert (attribute (name best-alcohol) (value harsh) (certainty .05)))
   (assert (attribute (name best-flavor) (value malty-sweet) (certainty .05)))
   ...)
```

Many rules assert the same value on the same axis; these duplicate attributes are folded into one before selection by the `combine-certainties` rule, which merges any two `attribute` facts that agree on `name` and `value` using the MYCIN combination function below.

Uncertainty Management. Reasoning under uncertainty is implemented with *certainty factors* ($CF \in [-1, 1]$). When two independent pieces of evidence assert the same value on the same axis with certainties x and y, they are combined by the classical MYCIN parallel-combination function:

$$CF(x,y) = \begin{cases} x + y - x\,y & x > 0,\ y > 0, \\ x + y + x\,y & x < 0,\ y < 0, \\ \dfrac{x+y}{1 - \min(|x|,|y|)} & \text{otherwise.} \end{cases} \tag{5.1}$$

Concordant positive evidence accumulates towards $+1$, concordant negative towards -1, and discordant evidence partially cancels. This is exactly the `combine-CFs` deffunction in `beerex.clp`:

```
(deffunction combine-CFs (?x ?y)
   (if (and (> ?x 0) (> ?y 0))
    then (bind ?c (- (+ ?x ?y) (* ?x ?y)))
    else (if (and (< ?x 0) (< ?y 0))
         then (bind ?c (+ (+ ?x ?y) (* ?x ?y)))
         else (bind ?c (/ (+ ?x ?y) (- 1 (min (abs ?x) (abs ?y)))))))
   ?c)
```

In the *CF* backend the recommendation is then scored by *accumulation*: the low-salience `generate-beers` rule matches each candidate beer's symbolic slots against the combined *best-** evidence and, for every match on `best-style`, `best-name`, `best-alcohol`, `best-color`, `best-flavor`, `best-fermentation` or `best-carbonation`, asserts a result `(attribute (name beer) ...)` carrying that piece of evidence's certainty. The final rule sorts the results by certainty and reports the top five.

Certainty factors versus fuzzy logic. The certainty-factor weights are assigned by the expert and are not quantified by the source guides; this arbitrariness, and the coarse ties it produces, motivate the fuzzy alternative, which replaces the hand-assigned certainties on the *measurable* axes with objective, data-grounded memberships.

Linguistic variables. The measurable axes are modelled as linguistic variables —colour over SRM, alcohol over ABV%, and a new bitterness axis over IBU— declared as `ling-var` facts with their universe of discourse, and each covered by overlapping linguistic terms encoded as `ling-term` facts. Each term is a *trapezoidal* membership function $a \le b \le c \le d$:

$$\mu_{a,b,c,d}(x) = \begin{cases} 0 & x \le a \text{ or } x \ge d, \\ \dfrac{x-a}{b-a} & a < x < b, \\ 1 & b \le x \le c, \\ \dfrac{d-x}{d-c} & c < x < d, \end{cases} \tag{5.2}$$

with $a{=}b$ giving a left shoulder and $c{=}d$ a right shoulder. The breakpoints are *derived from data*: crossing the dataset's symbolic labels with the guide's numeric ranges yields each term's empirical distribution, validated against the p25/median/p75 quantiles. Table 5.2 records the breakpoints currently in use (the `ling-term` facts of `beer-fuzzy.clp`).

Table 5.2: Data-grounded trapezoidal breakpoints (a, b, c, d) of the linguistic terms.

axis	source	terms (breakpoints a, b, c, d)			
colour	SRM	pale (0,0,6,9)	amber (6,9,11,14)	brown (11,14,21,27)	dark (21,27,50,50)
alcohol	ABV%	n.d. (2,2,4.5,5.2)	mild (4.5,5,6,6.6)	noticeable (6,6.6,8,8.6)	harsh (8,8.6,15,15)
bitterness	IBU	low (0,0,20,25)	medium (20,25,30,35)	high (30,35,70,70)	

Carbonation, by contrast, stays symbolic: the guide's CO_2 volumes do not separate *medium* from *high*, so a data-driven membership function would not be justified.

Defuzzification. Membership evaluation and defuzzification are provided by the reusable `fuzzy.clp` library, whose centroid (centre-of-gravity) routine integrates a piecewise-linear membership function *analytically*, segment by segment. For each trapezoidal segment between consecutive breakpoints (x_1, y_1) and (x_2, y_2), with $\mathrm{d}x = x_2 - x_1$, the area and the abscissa of its centroid are

$$A_i = \mathrm{d}x\,\frac{y_1 + y_2}{2}, \qquad \bar{x}_i = x_1 + \mathrm{d}x\,\frac{y_1 + 2y_2}{3\,(y_1 + y_2)}, \tag{5.3}$$

and the defuzzified value is the area-weighted mean of the segment centroids,

$$x^\star = \frac{\sum_i A_i \, \bar{x}_i}{\sum_i A_i}. \tag{5.4}$$

This is the `fuzzy-centroid` deffunction, and it reproduces the `moment-defuzzify` primitive of the original FuzzyCLIPS exactly:

```
(bind ?area (* ?dx (/ ?sy 2)))
(bind ?cx (+ ?x1 (* ?dx (/ (+ ?y1 (* 2 ?y2)) (* 3 ?sy)))))
(bind ?num (+ ?num (* ?area ?cx)))
(bind ?den (+ ?den ?area))
```

Scoring. In fuzzy mode the `beer-generate-fuzzy` bridge translates the combined *best-** evidence into weighted `desire` facts on the linguistic axes, `desire-cat` facts on the categorical slots and direct `suggest-name`/`suggest-style` suggestions. Each beer is ranked by a *normalised weighted-mean* satisfaction score: writing w_k for the weight of desire k and $m_k(\text{beer})$ for the beer's satisfaction of it —the trapezoidal membership $\mu(x)$ of the beer's measured value x for a continuous axis, or 1/0 for a categorical match—

$$S(\text{beer}) = \frac{\sum_k w_k \, m_k(\text{beer})}{\sum_k |w_k|}. \tag{5.5}$$

The denominator uses $|w_k|$ so that a beer with *missing data* on a desired axis is *penalised* (contributes 0 to the numerator while still counting in the denominator) rather than exempted. A further term boosts styles whose authoritative CraftBeer pairing matches a food named by the user: when the user named any food, a half-weighted `pairing-bonus` (1.0 on a textual hit, 0.0 otherwise) is added to both sums. The crisp value x fed to μ is the midpoint of the beer's measured range (`beer-axis-value`). Because the memberships are objective and continuous, the fuzzy ranking grades the candidates *finely* where the certainty-factor model can only produce coarse ties.

Explanation Module. A good expert system must justify its behaviour, and BeerEX does so in two complementary ways. The `get-explanation` deffunction builds a narrative from the `explanation-scenario`, `explanation-preference` and `explanation-*-meal` attributes accumulated during the dialogue, explaining *why* the chosen profile was inferred. When the fuzzy backend is active it appends a per-beer, data-grounded rationale: `beer-rationale` reports, for each recommended beer, which desires it satisfied and how strongly (e.g. "*noticeable alcohol (μ0.82), hoppy-bitter, matches its CraftBeer pairing*"), stored in `beer-why` facts. During the dialogue the user may additionally ask *why* a question is being posed (each `UI-state` carries a `why` text) or request *help*. All explanatory text is base English passed through the `T` deffunction, which overlays the optional Italian translation when `?*lang*` $\neq$ `en` (Section 5.5).

5.5 User Interface

The dialogue is a state machine, encoded in `dialog.clp` and shared by the CLI and the bot. A `UI-state` fact represents the current screen —its `display`, admissible `valid-answers`, `help`/`why` text and the `relation-asserted`— and a `state-list` keeps the current turn and the ordered `sequence` of visited turns. The `ask-question` rule presents a screen and *halts* the engine, returning control to the caller:

```
(defrule ask-question
   (declare (salience ?*medium-high-priority*))
   (UI-state (id ?id))
   ?f <- (state-list (sequence $?s&:(not (member$ ?id ?s))))
```

```
    =>
    (modify ?f (current ?id) (sequence ?id ?s))
    (halt))
```

Answering asserts a `(next ?id ?response)` fact; the `handle-next` rules advance the conversation, and `handle-prev` rewinds it —retracting the relation asserted at the revised turn (and refreshing the affected `random-question` / selection rules) so that working memory stays consistent after the user changes an earlier answer. The `translate-ui` rule localises each screen in place before it is shown when a non-English language is active.

The same machinery drives both front ends. The CLI (`BeerEX.clp`) implements a textual REPL with `ask-question`/`next-UI-state` deffunctions. The Telegram bot (`BeerEX.bot.py`) is built on python-telegram-bot v21 (async) and reaches the engine through clipspy. Crucially, each chat owns an *isolated* `clips.Environment`, fixing a serious flaw of the original single-environment bot in which concurrent users shared working memory:

```
class Session:
    """An isolated CLIPS environment driving one user's dialogue."""
    def __init__(self, backend=DEFAULT_BACKEND, lang=DEFAULT_LANG):
        self.backend = backend
        self.lang = lang
        self.env = clips.Environment()
        for f in CORE_FILES:
            self.env.load(_p(f))
        ...
        self.reset()
```

Sessions are kept in a `chat_id` → `Session` map (with a bounded, oldest-first eviction policy). Each turn renders the screen's `valid-answers` as inline keyboard buttons, with optional *Previous*, *Help*, *Why*, *Restart* and *Cancel* controls; help text embedding `_Label_(image-url)` links is turned into inline image buttons. The bot also exposes commands to switch the reasoning backend at runtime (`/cf`, `/fuzzy`) and the output language (`/en`, `/it`), each re-applying the corresponding global on `(reset)`; bot-side messages are localised through `Session.tr`, which calls the same `T` catalogue used by the engine. Adding a new language amounts to providing a new overlay file of `translation` facts, with no change to the engine or the knowledge base.

Chapter 6

Usage

This chapter describes how *CF* is started and how it is used in practice. The system exposes the same reasoning engine through two interfaces: a command-line REPL, useful for development and testing, and a Telegram bot, which is the intended everyday interface —usable at the pub, not in front of a computer. Both interfaces drive the same dialogue state machine and the same knowledge base, so a consultation conducted from the terminal and one conducted from the bot ask the very same questions and reach the very same recommendations. The chapter first explains how to start each interface, then enumerates the available commands, and finally walks through a number of worked sessions —a full consultation, a meal-pairing consultation, a comparison of the two reasoning backends, the *Help* and *Why* facilities, and the revision and cancellation of answers— each illustrated with a faithful transcript.

6.1 Starting the system

The reasoning engine runs from the terminal with a CLIPS 6.4.x interpreter. The project does not assume a system-wide CLIPS: the helper script `scripts/build-clips.sh` builds CLIPS 6.4.2 locally into `~/.local/bin`, after which the engine is launched by loading its start-up file `BeerEX.clp`:

```
bash scripts/build-clips.sh # builds CLIPS 6.4.2 into ~/.local/bin
clips -f BeerEX.clp
```

The start-up file loads the core templates and globals, the fuzzy backend, the per-style numeric ranges and profiles, the dialogue state machine and, finally, the question and knowledge rule files; it then performs a `(reset)` and `(run)` and hands control to the dialogue loop, which prints the first question and waits for the user's answer. From then on the program asks the questions one at a time, accepting either one of the displayed options or one of the *help*, *why*, *prev* (back) and *cancel* control words.

The Telegram bot, by contrast, needs no system CLIPS: the `clipspy` binding bundles its own interpreter. The bot is built on *python-telegram-bot* v21 and keeps one isolated CLIPS environment per chat, so that concurrent users never share working memory. With a token obtained from *@BotFather*, the bot is started either through Docker or directly with Python 3.12:

```
cp .env.example .env # put your @BotFather TOKEN in .env
docker compose up --build # recommended

# or, for a local run:
git submodule update --init # fetch the fuzzy library
python3.12 -m venv .venv && . .venv/bin/activate
pip install -r requirements.txt
export TOKEN=<your-telegram-bot-token>
python BeerEX.bot.py
```

Once the process is polling, the user opens the chat and types `/start`. The bot greets the user by name and prints a short presentation, after which a consultation can be started with `/new`. The options for each question are offered as a reply keyboard, together with the *Help*, *Why*, *Previous* and *Cancel* buttons where applicable; when a recommendation has been produced, the keyboard offers instead the *Previous*, *Restart* and *Cancel* buttons.

6.2 Commands

The bot recognises a small set of slash commands, which control the session as a whole, and a set of in-dialogue buttons, which drive the question/answer interaction. The slash commands are the following:

- `/start` — greets the user and prints a short description of how the system works;
- `/new` — starts a new consultation, resetting the working memory and asking the first question;
- `/cf` — selects the Certainty-Factors reasoning backend (the classic MYCIN-style evidence combination);
- `/fuzzy` — selects the fuzzy-logic reasoning backend, which scores every style with data-grounded membership functions;
- `/en` — sets the output language to English (the base language);
- `/it` — sets the output language to the Italian overlay, which falls back to English wherever a string is not yet translated.

The `/cf`, `/fuzzy`, `/en` and `/it` commands do not affect the consultation already in progress: they record the requested backend or language on the session and take effect at the next `/new`. English is always the base; the commands themselves are control tokens and are never localised, only the questions, recommendations and explanations that the user reads are.

During a consultation the following in-dialogue controls are available, exposed as keyboard buttons (and as the corresponding control words `help`, `why`, `prev`, `restart` and `cancel` on the command line):

- *Help* — shows an extended description of the current question's options; offered only when the question carries a help text;
- *Why* — explains why the current question is being asked; offered only when the question carries a rationale;
- *Previous* — goes back to the preceding question, retracting the last answer so that it can be revised; offered only once at least one question has been answered;
- *Restart* — abandons the current line of questioning and begins a fresh consultation; offered on the final recommendation screen;
- *Cancel* — closes the consultation and clears the working memory.

6.2.1 A worked consultation

A full consultation begins with a block of profiling questions —asked in random order, since they are independent of one another— covering the drinker's profile (age, regular-drinker status, smoking habit, season) and their personal preferences (colour, alcohol, carbonation, flavour). Once these are known, the dialogue switches to a depth-first chain of meal questions. The following command-line transcript drives the profiling questions to the point where a recommendation is produced; the questions and their options are exactly those defined in the knowledge base.

```
How old are you?
1.18-23 2.24-29 3.>=30 4.why 5.cancel
Answer: 3

Are you a regular beer drinker?
1.yes 2.no 3.why 4.cancel 5.prev
Answer: yes

Do you usually smoke while drinking?
1.yes 2.no 3.why 4.cancel 5.prev
Answer: no

Is it autumn, spring, summer or winter?
1.autumn 2.spring 3.summer 4.winter 5.why 6.cancel 7.prev
Answer: winter

Do you generally prefer pale, amber, brown or dark beer?
1.pale 2.amber 3.brown 4.dark 5.don't mind 6.cancel 7.prev
Answer: dark

Do you generally prefer to drink low, mild, high or very high alcoholic drinks?
1.low 2.mild 3.high 4.very high 5.don't mind 6.cancel 7.prev
Answer: high

Do you generally prefer to drink low, medium or high carbonated drinks?
1.low 2.medium 3.high 4.don't mind 5.cancel 6.prev
Answer: medium

What kind of flavor do you generally prefer?
1.clean 2.sweet 3.bitter 4.roasty 5.fruity 6.spicy 7.sour 8.don't mind 9.cancel 10.prev
Answer: roasty
```

At this point the drinker's profile is complete and the dialogue moves on to the meal. For the purposes of this example the drinker declares that no particular dish is involved; the engine combines the accumulated evidence and prints the recommendation. With the fuzzy backend active, the result block names each style, its serving hint and its graded certainty, and is followed by the narrative explanation of the inferred profile together with a per-beer rationale:

```
Is the main meal pizza, entre, cheese or dessert?
1.pizza 2.entre 3.cheese 4.dessert 5.other 6.cancel 7.prev
Answer: other

 Done. I have selected these beer styles for you...

 German-Style Schwarzbier   45-50F   Flute with certainty 71%
 German-Style Dunkel   45-50F   Vase with certainty 66%
 American Stout   50-55F   Tulip with certainty 63%

... for the following reasons:

Your palate could be very demanding: I bet you would try out something
peculiar. Frosty days are perfect to taste something sweet or roasty, that
```

```
strongly change your palate. Also, high alcoholic beers can heat you up.
You prefer a dark beer. You chose a high alcoholic beer. It's a medium
carbonated beer. You desire a beer with a roasty flavor.

 Why these beers:
 German-Style Schwarzbier: dark colour (1.00), noticeable alcohol
(0.85), medium carbonation, dark-roasty flavour.
```

The narrative explanation is assembled, in order, from the scenario, preference, main-meal and specific-meal rationales accumulated during the consultation, so the user can always trace the recommendation back to the answers that produced it. Under the fuzzy backend a final *Why these beers* paragraph reports, for each recommended style, the data-grounded membership values that earned it its place in the ranking.

6.2.2 Meal pairing

When the drinker names a dish, the dialogue descends a meal-specific chain of questions and the recommendation becomes pairing-driven: the authoritative CraftBeer food pairings stored in the knowledge base are consulted, and styles whose pairing matches one of the foods the user named are rewarded. The transcript below assumes the profiling questions have already been answered as in the previous session, and picks up at the main-meal question, following the *pizza* branch to a specific topping:

```
Is the main meal pizza, entre, cheese or dessert?
1.pizza 2.entre 3.cheese 4.dessert 5.other 6.cancel 7.prev
Answer: pizza

Is the pizza topping classic, meat, vegetables, cheese or other?
1.classic 2.meat 3.vegetables 4.cheese 5.other 6.why 7.cancel 8.prev
Answer: meat

Is the meat spicy?
1.yes 2.no 3.why 4.cancel 5.prev
Answer: no

 Done. I have selected these beer styles for you...

 American IPA   50-55F   Tulip with certainty 78%
 English-Style IPA   45-50F   Nonic Pint with certainty 74%
 Imperial IPA   50-55F   Tulip with certainty 70%

... for the following reasons:

You're eating a pizza with meat: the best way to enjoy it is pair that with
beers with a bitter flavor, because of the bitterness of the hops that
pleasantly cleanses the mouth.
```

The non-spicy meat topping makes the knowledge base assert a strong preference for a *hoppy-bitter* flavour, which the fuzzy backend translates into a desire for high bitterness on the IBU axis; the hop-forward styles therefore rise to the top of the ranking. Had the meat been declared spicy, the opposite evidence would have been asserted —a malty-sweet flavour to neutralise the capsaicin— and the ranking would have shifted towards the malt-forward styles instead.

6.2.3 Switching the backend

The same profile can be evaluated under either reasoning backend, and the two produce visibly different rankings. The Certainty-Factors backend combines the discrete evidence asserted by the knowledge base in the MYCIN style: a style is scored by the certainty factors that vote

for it, which yields a coarse ranking in which several styles frequently share the same certainty and so tie. The fuzzy backend, by contrast, scores every style by a normalised weighted mean of data-grounded membership values —colour against SRM, alcohol against ABV, bitterness against IBU— so that ties are broken by how well each style actually fits the desired profile, and the certainties come out as graded percentages rather than as a handful of repeated values.

Selecting the backend is a matter of one slash command, which takes effect at the next `/new`:

```
/cf
Backend set to cf. Press /new to start.
```

Running the same answers as in the worked consultation above under the two backends illustrates the difference. Under `/cf` the top of the ranking tends to be a block of styles sharing the certainty contributed by the dominant rule, for example:

```
Done. I have selected these beer styles for you...

German-Style Schwarzbier with certainty 50%
German-Style Dunkel with certainty 50%
American Stout with certainty 50%
```

Under `/fuzzy` the same three styles are separated, because the membership scores grade how closely each one matches the requested dark colour, noticeable alcohol and roasty flavour, and the serving hint and per-beer rationale are added:

```
Done. I have selected these beer styles for you...

German-Style Schwarzbier   45-50F   Flute with certainty 71%
German-Style Dunkel   45-50F   Vase with certainty 66%
American Stout   50-55F   Tulip with certainty 63%
```

The Certainty-Factors ranking is therefore useful as a quick, transparent view of which rules fired, while the fuzzy ranking is preferable whenever a finer, fully ordered recommendation is wanted. Because both backends feed the same final formatting stage, the explanation and the result layout are identical in structure; only the certainties, the ordering and the presence of the fuzzy rationale differ.

6.2.4 Help and Why

Every question may carry two optional pieces of supporting text: a *help* text, which explains the meaning of the options, and a *why* text, which justifies the question. When present, they are offered as the *Help* and *Why* buttons (or the `help` and `why` control words on the command line) and do not advance the dialogue: after the text is shown, the same question is asked again.

The *Why* facility is illustrated on the age question. Requesting `why` prints the rationale and re-poses the question:

```
How old are you?
1.18-23 2.24-29 3.>=30 4.why 5.cancel
Answer: why

Younger people tend to prefer less intense beers.

How old are you?
1.18-23 2.24-29 3.>=30 4.why 5.cancel
Answer: 1
```

The *Help* facility is illustrated on the cheese-style question, one of the few questions rich enough to carry a help text describing each option in detail. Requesting `help` prints the extended description of the cheese families before the question is asked again:

```
Is the cheese style fresh, semi-soft, firm/hard, blue, natural rind
or washed rind?
1.fresh 2.semi-soft 3.firm/hard 4.blue 5.natural rind 6.washed rind
7.don't know 8.help 9.why 10.cancel 11.prev
Answer: help

 Fresh cheeses have not been aged, or are very slightly cured. These
cheeses have a high moisture content and are usually mild and have a very
creamy taste and soft texture.
 Semi-soft cheeses have a smooth, generally, creamy interior with little
or no rind. [ ... ]
```

In the Telegram interface the help text is rendered with Markdown; where it embeds an image link, the bot turns the link into an inline button that sends the corresponding picture when pressed.

6.2.5 Going back and cancelling

The *Previous* control lets the user revise an answer. Going back retracts the answer given to the current question and, with it, any conclusion derived from that answer: the dialogue state machine removes the fact asserted for the question and re-fires the affected rules, so that working memory remains consistent with the answers actually in force. The transcript below shows a drinker who answered *pale* to the colour question, went back, and changed the answer to *dark*:

```
Do you generally prefer pale, amber, brown or dark beer?
1.pale 2.amber 3.brown 4.dark 5.don't mind 6.cancel 7.prev
Answer: pale

Do you generally prefer to drink low, mild, high or very high alcoholic drinks?
1.low 2.mild 3.high 4.very high 5.don't mind 6.cancel 7.prev
Answer: prev

Do you generally prefer pale, amber, brown or dark beer?
1.pale 2.amber 3.brown 4.dark 5.don't mind 6.cancel 7.prev
Answer: dark
```

Because the colour evidence asserted for *pale* is retracted when the user goes back, the subsequent recommendation reflects only the corrected *dark* preference, with no residual trace of the abandoned answer. The same mechanism applies even on the final recommendation screen: going back from there retracts the last meal answer and re-poses it, so the user can explore alternative meal choices without restarting.

Finally, the *Cancel* control closes the consultation. On the command line it terminates the interpreter; in the bot it clears the chat's working memory, removes the keyboard and bids the user farewell:

```
Answer: cancel
Bye! I hope we can talk again some day.
```

After cancelling, a fresh consultation can always be started again with `/new` (or by relaunching the command-line interpreter).

Chapter 7

Validation

BeerEX is validated with a small regression suite that exercises every layer of the system: the pure-CLIPS fuzzy library is checked against an authentic FuzzyCLIPS engine used as an oracle; the beer fuzzy backend is exercised end to end by a self-checking CLIPS suite; the two uncertainty backends are compared on a single fixed profile; and the production engine path is driven through a full dialogue. All suites are reproducible from the repository root and are wired into continuous integration.

7.1 Method

The validation rests on four complementary activities.

(a) Fuzzy library vs the original engine. The pure-CLIPS fuzzy library (`third_party/FuzzyCLIPS/fuzzy`) is validated by *differential testing* against the authentic FuzzyCLIPS 6.10d engine, built from source and used as an oracle. The runner `diff_test.sh` computes each quantity twice —once with the library on stock CLIPS 6.4.x, once with the native engine— and compares the two. Four families are covered: centroid *defuzzification* on piecewise-linear sets, the *set operators* (union, intersection, complement), full *Mamdani* aggregation and defuzzification reproducing the canonical tipper consequents, and the curved (sampled) membership functions (S, Z and π). Piecewise-linear cases are required to match to a tight tolerance ($\varepsilon = 10^{-4}$); the sampled curves are allowed a small grid tolerance ($\varepsilon = 0.2$).

(b) Scoring and integration suite. The CLIPS suite `tests/test-scoring.clp` loads the real fuzzy backend (`beer-ranges.clp`, `beer-data.clp`, `beer-fuzzy.clp`) over the authentic beer-style facts and runs a sequence of self-checking assertions on the data-grounded scoring: that a "pale" query ranks pale styles first and "dark" ranks dark styles; that the worked example *pale (0.8)+bitter (0.6)* yields the expected exact scores and ordering; that catch-all styles with no characteristic profile are not rewarded; that the *CF*-to-fuzzy bridge turns `best-*` attributes into ranked beer attributes; that a direct suggestion boosts the suggested beer; and that an authoritative meal pairing boosts a beer matched to a named food, with a serving hint attached.

(c) Certainty factors vs fuzzy. The differential script `tests/test-cf-vs-fuzzy.sh` drives the real selection rules end to end, once per backend, on a *single fixed profile*: a preference for pale colour and bitter flavour (*CF* 0.2 each) together with a direct pairing suggestion for American IPA (*CF* 0.9). The dialogue rules are removed so a controlled end-state can be injected, and the recommendations of the `cf` and `fuzzy` backends are printed side by side, exposing the qualitative difference between the two ranking strategies.

(d) Engine smoke test and CI. The Python test `tests/test_engine.py` drives the bot's per-user CLIPS `Session` over `clipspy` —the real production engine path— through a full dialogue, answering the first valid option at each step until a recommendation is reached. It checks that both backends conclude, that the fuzzy recommendation carries a data-grounded explanation, that the Italian overlay resolves while English remains the base, and that two sessions never share a CLIPS environment. Continuous integration runs the linter and these tests on every push.

7.2 Cases and results

Table 7.1 reports the recommendation rankings produced by the two backends on the fixed profile of activity (c). The *CF* backend produces a single strong top hit followed by a flat tie —three styles at 20%— whereas the fuzzy backend grades the candidates finely, because its membership values are data-grounded.

Table 7.1: Certainty-factor vs fuzzy ranking on the fixed pale + bitter profile.

Style	**cf**	**fuzzy**
American IPA	92%	84%
American Pale Ale	20%	—
English-Style Bitter	20%	—
English-Style IPA	20%	—
Bohemian-Style Pilsener	—	100%
Belgian-Style Golden Strong Ale	—	87%
German-Style Pilsener	—	75%

Table 7.2 summarises the four validation suites and their measured outcome. All suites pass.

Table 7.2: Validation suites and results.

Suite	**What it checks**	**Result**
`diff_test.sh`	fuzzy library vs FuzzyCLIPS 6.10d oracle	12/12
`test-scoring.clp`	beer fuzzy backend, scoring & integration	16/16
`test-cf-vs-fuzzy.sh`	cf vs fuzzy on the fixed profile	6/6
`test_engine.py`	clipspy dialogue + session isolation	PASS

7.3 Findings

- On the piecewise-linear cases the library reproduces the FuzzyCLIPS oracle *exactly* —to about thirteen significant digits— on defuzzification (e.g. a left triangle at 16.66667, a centred one at 50.00000, a trapezoid at 30.00000), the set operators (50.97826 union, 40.83333 intersection, 58.88889 complement) and the full Mamdani pipeline (21.53419 on the tipper). The curved (sampled) membership functions converge to the oracle within the grid tolerance (e.g. $S(20, 80)$ at 73.498 vs 73.406). The whole suite reports 12/12 matches.
- The scoring suite confirms the data-grounded behaviour end to end (16/16): "pale" ranks a pale style first and "dark" a dark one; the worked example *pale (0.8)+bitter (0.6)* yields the exact scores (0.7857 for German-Style Pilsener, 0.4286 for Imperial IPA) with Pilsener correctly ahead; catch-all styles with no characteristic axis score 0 and are not rewarded;

a direct suggestion boosts the suggested beer; and an authoritative pairing boosts a beer matched to a named food, with a serving hint (temperature and glassware) attached.

- The cf-vs-fuzzy run makes the qualitative difference concrete. The *CF* backend collapses into a strong leader (American IPA at 92%) followed by a coarse tie of three styles at 20%, providing no basis to order them. The fuzzy backend instead grades the field finely (100 / 87 / 84 / 75%) and surfaces the per-beer rationale (e.g. "pale colour μ1.00, high bitterness μ1.00"). Fuzzy ranking is preferred wherever the source beer guides specify *quantitative* style ranges but not certainty-factor weights: the membership values are computed from those ranges, so they are data-grounded rather than hand-tuned, and they break ties the *CF* backend cannot.
- The engine smoke test confirms that the production path (`clipspy`) drives a complete dialogue to a recommendation under both backends, that the fuzzy recommendation carries its data-grounded explanation, and that per-user sessions are isolated —each holds a distinct CLIPS environment, so concurrent consultations cannot interfere. The same tests gate every push through continuous integration.

References

[1] G. Riley. *CLIPS: A Tool for Building Expert Systems*. https://www.clipsrules.net/, 2025 (version 6.4.2).

[2] R. Orchard. *FuzzyCLIPS Version 6.10d User's Guide*. Integrated Reasoning Group, National Research Council of Canada, 2004.

[3] E. H. Shortliffe and B. G. Buchanan. *A model of inexact reasoning in medicine*. Mathematical Biosciences, 23(3–4):351–379, 1975.

[4] L. A. Zadeh. *Fuzzy sets*. Information and Control, 8(3):338–353, 1965.

[5] E. H. Mamdani and S. Assilian. *An experiment in linguistic synthesis with a fuzzy logic controller*. International Journal of Man-Machine Studies, 7(1):1–13, 1975.

[6] Brewers Association / CraftBeer.com. *Beer Styles Study Guide*, 2017.

[7] A. Dulye and J. Herz. *CraftBeer.com Beer & Food Course*. Brewers Association.

[8] M. Cerrato. *clipspy: CLIPS Python bindings*. https://github.com/noxdafox/clipspy.